

Chapter 2 JavaScript Essentials

CHAPTER OBJECTIVES

In this chapter, we'll work through two of the essential building blocks of JavaScript: variables and operators. We'll begin with variables — what they are and the various data types they can hold. These fundamental building blocks let us store and manipulate values within our scripts, giving them dynamic behavior.

Once variables are out of the way, we'll move on to operators. We'll cover arithmetic, assignment, comparison, and logical operators. Operators allow us to modify variables or extract information about them, enabling us to perform calculations based on factors such as user input.

Along the way, we'll cover:

- Variables
 - Primitive data types
 - Analyzing and modifying data types
 - Operators

Note: exercise, project, and self-check quiz answers can be found in the Appendix.

Variables

Variables are typically the very first building block introduced when learning a new programming language. A variable is a value in your code that can represent something different each time the program runs. Here are two example variables:

```
firstname = "Maaike";  
x = 2;
```

These values can also be reassigned while the program is executing:

```
firstname = "Edward";  
x = 7;
```

Without variables, a piece of code would perform identically every single time it ran. While that's occasionally useful, code becomes far more powerful when it can behave differently from one run to the next — and that flexibility is exactly what variables provide.

Declaring variables

The first time you create a variable, you declare it — and declaring a variable requires one of three special keywords: `let`, `var`, or `const`. We'll discuss the distinctions between these shortly. After the initial declaration, subsequent uses of the variable refer to it by name alone, simply to assign a new value:

```
let firstname = "Maria";  
firstname = "Jacky";
```

In our examples so far, we've been assigning values directly within the code itself — this is referred to as a "hardcoded" value, since it's fixed in the script rather than supplied dynamically from some outside source. In real-world code, you'll rarely hardcode values this way; more often, values come from external sources such as an input field on a web page, a database, or another piece of code that invokes your script. This is precisely what makes scripts adaptable — they can respond to new information without requiring any rewriting.

We've now established just how powerful variables can be. For the moment, we'll continue hardcoding values into our examples, meaning they won't actually vary unless a developer manually edits the program. Soon, though, we'll see how to populate variables from external sources.

`let`, `var`, and `const`

A variable declaration consists of three components: a keyword (`let`, `var`, or `const`), a name, and a value. Here are examples using each keyword:

```
let nr1 = 12;  
var nr2 = 8;  
const PI = 3.14159;
```

`let` and `var` are both used for variables that may be reassigned a new value at some point as the program runs. The distinction between them is somewhat nuanced and relates to scope.

`var` has global scope, while `let` has block scope. The global scope of `var` means it can be referenced anywhere throughout the entire script. By contrast, the block scope of `let` restricts the variable to the specific block of code in which it was defined. A code block always begins with `{` and ends with `}`, which is how you can identify one.

`const`, by comparison, is used for variables whose value is assigned exactly once and never changed — the value of `pi`, for instance. Attempting to reassign a `const` variable produces an error:

```
const someConstant = 3;  
someConstant = 4;
```

This results in:

```
Uncaught TypeError: Assignment to constant variable.
```

Throughout most of our examples, we'll be using `let` — for now, simply trust that `let` is the right default choice in most situations.

If the preceding discussion of scope made sense to you, great — but it's completely fine if it didn't sink in fully just yet. You'll come to understand it naturally as you continue working through the book.

Naming variables

A few naming conventions apply when working with variables:

- Variable names begin with a lowercase letter and should be descriptive. If a variable holds someone's age, name it `age` rather than something opaque like `x` — this way, your code remains self-explanatory when you (or someone else) revisits it later.
- Variable names cannot contain spaces, though underscores are permitted. A space would prevent JavaScript from recognizing the name as a single variable.

We'll be using camelCase throughout this book. This means that when a variable name needs multiple words, the first word is lowercase and each subsequent word begins with a capital letter — for example, `ageOfBuyer`.

Whatever convention is used at your workplace, the most important thing is consistency. When naming follows a uniform pattern throughout a codebase, the result is cleaner, more readable code that's far easier to modify later.

A variable's value can be just about anything. Let's begin with the simplest category: primitives.

Primitive data types

Now that we understand what variables are and why we need them, it's time to examine the different kinds of values they can hold. JavaScript is a loosely typed language, meaning the type of a value is inferred automatically rather than declared explicitly. If you assign the value `5` to a variable, JavaScript automatically treats it as a number.

A distinction exists between primitive data types and more complex data types. This chapter focuses on primitives — relatively simple data structures that, for our purposes, simply hold

a value paired with a type. JavaScript defines seven primitive types: String, Number, BigInt, Boolean, Symbol, undefined, and null. We'll examine each in turn.

String

A string stores a sequence of characters — text. There are three ways to declare a string in JavaScript:

- Double quotes
- Single quotes
- Backticks — special template strings that allow variables to be embedded directly

Single and double quotes function similarly:

```
let singleString = 'Hi there!';  
let doubleString = "How are you?";
```

The key difference is that single quotes can appear as literal characters inside a double-quoted string, and vice versa. If a string is declared with single quotes, it ends the instant a second single quote is encountered — even mid-word. The following, for example, produces an error, because the string terminates at the apostrophe inside "let's":

```
let funActivity = 'Let's learn JavaScript';
```

JavaScript would interpret 'Let' as the complete string, leaving the remaining characters uninterpretable. Switching to double quotes resolves this, since the parser is now looking for a closing double quote rather than a single quote:

```
let funActivity = "Let's learn JavaScript";
```

The same principle applies in reverse — a double quote embedded in a double-quoted string causes the same problem:

```
let question = "Do you want to learn JavaScript? "Yes!"";
```

Use whichever quoting style you prefer, unless you're working within an existing codebase that has already established a convention — in which case, consistency takes priority.

In a backtick-delimited string, variable values can be embedded directly using a special syntax:

```
let language = "JavaScript";
let message = `Let's learn ${language}`;
console.log(message);
```

Variables are referenced inside `${...}`. This somewhat unusual syntax is deliberate — it's distinctive enough that it won't accidentally appear in ordinary text. In this example, the console output is:

```
Let's learn JavaScript
```

The `language` variable is substituted with its value, `JavaScript`.

Escape characters

Suppose you need to include double quotes, single quotes, and backticks all within the same string — none of our three quoting methods alone can handle this directly. The solution is the escape character: the backslash, which tells JavaScript "interpret the next character literally, not according to its usual meaning."

```
let str = "Hello, what's your name? Is it \"Mike\"?";
console.log(str);
let str2 = 'Hello, what\'s your name? Is it "Mike"?';
console.log(str2);
```

This logs:

```
Hello, what's your name? Is it "Mike"?
Hello, what's your name? Is it "Mike"?
```

In both cases, the quote marks inside the string are preserved without prematurely terminating the string, because the preceding backslash redefines them as literal characters rather than string delimiters.

The escape character serves additional purposes too. `\n` inserts a line break, and `\\` inserts a literal backslash character:

```
let str3 = "New \nline.";
let str4 = "I'm containing a backslash: \\!";
console.log(str3);
console.log(str4);
```

Output:

```
New  
line.  
I'm containing a backslash: \!
```

There are additional escape sequences beyond these, but we'll set those aside for now and return to primitive data types — specifically, numbers.

Number

The number data type represents, unsurprisingly, numbers. Many languages distinguish sharply between different numeric types, but JavaScript's designers chose a single unified type — **number** — implemented as a 64-bit floating-point value. This allows it to represent very large numbers, signed and unsigned values, decimals, and more.

The number type can represent integers (4, 89), decimals, exponential notation, and octal, hexadecimal, and binary representations:

```
let intNr = 1;  
let decNr = 1.5;  
let expNr = 1.4e15;  
let octNr = 0o10; //decimal version would be 8  
let hexNr = 0x3E8; //decimal version would be 1000  
let binNr = 0b101; //decimal version would be 5
```

Don't worry if octal, hexadecimal, or binary notation is unfamiliar — these are simply alternative ways numbers are sometimes represented in computer science. The key takeaway is that all of these values are, fundamentally, of type **number**. Integers like 3434 or -111 are numbers, floating-point values like 45.78 are numbers, and binary representations like 0b100 (equivalent to 4) are numbers as well.

The number type is by far the most commonly used numeric type — but for certain specialized cases, an even larger number may be required.

BigInt

The standard number type has a representable range between $2^{53} - 1$ and $-(2^{53} - 1)$. When values outside this range are needed, BigInt provides the solution. A BigInt literal is recognized by the trailing **n**:

```
let bigNr = 90071992547409920n;
```

Let's see what happens when mixing a BigInt with a regular number in a calculation:

```
let result = bigNr + intNr;
```

This produces:

```
Uncaught TypeError: Cannot mix BigInt and other types, use explicit conversions
```

The error message is unambiguous: `BigInt` and regular numbers cannot be directly combined in arithmetic operations. This is worth remembering when working with `BigInt` — operations involving a `BigInt` require both operands to be `BigInts`.

Boolean

The Boolean type holds exactly two values: `true` and `false`, with nothing in between. Booleans appear constantly throughout JavaScript code, particularly as the result of expressions that evaluate to true or false:

```
let bool1 = false;  
let bool2 = true;
```

Booleans are ideal for representing on/off or yes/no states — for instance, whether an object has been deleted:

```
let objectIsDeleted = false;
```

or whether a light is currently switched on:

```
let lightIsOn = true;
```

These two variables respectively indicate that the object in question has not been deleted, and that the light is on.

Symbol

Symbol is a relatively recent addition introduced in ES6 (recall from Chapter 1 that ECMAScript 6 is also referred to as ES6). A Symbol is useful when you need two variables to be guaranteed unequal even if their value and type appear identical. Compare the following string declarations to the equivalent Symbol declarations:

```
let str1 = "JavaScript is fun!";
let str2 = "JavaScript is fun!";
console.log("These two strings are the same:", str1 === str2);

let sym1 = Symbol("JavaScript is fun!");
let sym2 = Symbol("JavaScript is fun!");
console.log("These two Symbols are the same:", sym1 === sym2);
```

Output:

```
These two strings are the same: true
These two Symbols are the same: false
```

In the first case, JavaScript considers the strings identical, since they share both the same value and the same type. In the second case, each Symbol is inherently unique — despite carrying the same descriptive text, they are not considered equal. Symbols turn out to be extremely useful as unique object property keys, which we'll explore in Chapter 3, "JavaScript Multiple Values."

Undefined

JavaScript reserves a special data type, `undefined`, specifically for a variable that has been declared but never assigned a value:

```
let unassigned;
console.log(unassigned);
```

Output:

```
Undefined
```

You can also explicitly assign `undefined` as a value, but it's important to know that this is generally considered poor practice:

```
let terribleThingToDo = undefined;
```

While technically valid, this approach causes problems — particularly when checking equality between variables. If one variable was never assigned a value (and is therefore automatically `undefined`) and another was deliberately set to `undefined`, JavaScript will treat them as equal, even though their meanings are conceptually quite different. This could, for example, cause an unrelated pair of empty values — like someone's pet and their surname — to register as "equal" purely because both happen to be `undefined`.

null

The previous problem can be addressed using a final primitive type: `null`. `null` is a special value used to indicate that a variable is deliberately empty or its value is intentionally unknown. Note that `null` must always be written in lowercase:

```
let empty = null;
```

To resolve the issue described above, deliberately assigning `null` (rather than `undefined`) avoids the unintended equality problem:

```
let terribleThingToDo = undefined;
let lastName;
console.log("Same undefined:", lastName === terribleThingToDo);

let betterOption = null;
console.log("Same null:", lastName === betterOption);
```

Output:

```
Same undefined: true
Same null: false
```

This demonstrates the issue clearly: an automatically `undefined` variable (`lastName`) and a deliberately `undefined` variable (`terribleThingToDo`) are considered equal — a misleading outcome. By contrast, `lastName` and `betterOption` (explicitly set to `null`) are correctly recognized as not equal, since one is genuinely unassigned and the other is deliberately marked empty.

Analyzing and modifying data types

Having covered the primitive data types, JavaScript provides a number of built-in methods that help address common challenges involving them. Built-in methods are predefined pieces of functionality you can use without writing the underlying logic yourself. We've already encountered one such method: `console.log()`.

Working out the type of a variable

Determining a variable's data type — particularly with `null` and `undefined` — can sometimes be tricky. The `typeof` operator returns a variable's type. It can be used either with a space before the variable, or with the variable enclosed in parentheses:

```
testVariable = 1;
variableTypeTest1 = typeof testVariable;
variableTypeTest2 = typeof(testVariable);
console.log(variableTypeTest1);
console.log(variableTypeTest2);
```

Both forms output `number`. Parentheses aren't strictly required, since `typeof` is technically an operator rather than a method like `console.log()` — though using parentheses can sometimes improve readability. Here it is applied across multiple types:

```
let str = "Hello";
let nr = 7;
let bigNr = 12345678901234n;
let bool = true;
let sym = Symbol("unique");
let undef = undefined;
let unknown = null;

console.log("str", typeof str);
console.log("nr", typeof nr);
console.log("bigNr", typeof bigNr);
console.log("bool", typeof bool);
console.log("sym", typeof sym);
console.log("undef", typeof undef);
console.log("unknown", typeof unknown);
```

Output:

```
str string
nr number
bigNr bigint
bool boolean
```

```
sym symbol
undef undefined
unknown object
```

There's one notable anomaly: `typeof null` returns `"object"`, even though `null` is genuinely a primitive value, not an object. This is a long-standing bug in JavaScript, present

since the language's earliest days, and it can no longer be fixed without breaking backward compatibility across the web. It's not something you need to worry about in your own programs — just be aware that it exists, since it isn't going away.

Converting data types

JavaScript variables can change type, and JavaScript sometimes performs this conversion automatically. Consider:

```
let nr1 = 2;  
let nr2 = "2";  
console.log(nr1 * nr2);
```

Here, a number is multiplied by a string. Rather than throwing an error (as many other languages would), JavaScript first attempts to convert the string into a number. Since this conversion succeeds, the multiplication proceeds as if both operands were numbers, logging `4` to the console.

This automatic behavior, however, can be a source of subtle bugs. Consider:

```
let nr1 = 2;  
let nr2 = "2";  
console.log(nr1 + nr2);
```

This logs `"22"`. The `+` operator can also perform string concatenation, and in this case JavaScript converts the number into a string rather than the other way around, joining `"2"` and `"2"` into `"22"`.

Fortunately, we don't have to rely entirely on JavaScript's implicit conversion rules — explicit conversion functions are available: `String()`, `Number()`, and `Boolean()`.

`String()` converts virtually any value — including `undefined` and `null` — into a string representation enclosed in quotes.

`Number()` attempts to convert a value into a number. When a logical conversion isn't possible, the result is `NaN` ("not a number").

`Boolean()` converts a value into a Boolean. The result is `true` for nearly everything, except `null`, `undefined`, the number `0`, an empty string, and `NaN`.

```
let nrToStr = 6;
nrToStr = String(nrToStr);
console.log(nrToStr, typeof nrToStr);

let strToNr = "12";
strToNr = Number(strToNr);
console.log(strToNr, typeof strToNr);

let strToBool = "any string will return true";
strToBool = Boolean(strToBool);
console.log(strToBool, typeof strToBool);
```

Output:

```
6 string
12 number
true boolean
```

Some conversions produce results that aren't immediately intuitive:

```
let nullToNr = null;
nullToNr = Number(nullToNr);
console.log("null", nullToNr, typeof nullToNr);

let strToNr = "";
strToNr = Number(strToNr);
console.log("empty string", strToNr, typeof strToNr);
```

Output:

```
null 0 number
empty string 0 number
```

Both `null` and an empty string convert to the number `0` — a deliberate design decision in JavaScript that can occasionally be useful, for instance when converting potentially empty or null strings to a default numeric value of zero.

Now consider:

```
let strToNr2 = "hello";
strToNr2 = Number(strToNr2);
console.log(strToNr2, typeof strToNr2);
```

Output:

```
NaN number
```

Any string that cannot be meaningfully reinterpreted as a number evaluates to **NaN**.

And:

```
let strToBool2 = "false";
strToBool2 = Boolean(strToBool2);
console.log(strToBool2, typeof strToBool2);

let strToBool = "";
strToBool = Boolean(strToBool);
console.log(strToBool, typeof strToBool);
```

Output:

```
true boolean
false boolean
```

Remarkably, any non-empty string — including the literal string **"false"** — converts to **true**. Only an empty string, **null**, and **undefined** convert to **false**.

One final example:

```
let nr1 = 2;
let nr2 = "2";
console.log(nr1 + Number(nr2));
```

This logs **4**. Because the string is explicitly converted to a number before the addition takes place, the **+** operator performs arithmetic addition rather than string concatenation. The next section of this chapter explores operators in much greater depth.

Practice exercise 2.1

What are the types of the variables listed below? Verify your answers using **typeof** and log the results to the console:

```
let str1 = 'Laurence';  
let str2 = "Sveki";  
let val1 = undefined;  
let val2 = null;  
let myNum = 1000;
```

Operators

Having explored several data types and how to convert between them, it's time to turn to the next major building block: operators. Operators let us work with variables — modifying them, performing calculations, and comparing them. They're called operators because they operate on our variables.

Arithmetic operators

Arithmetic operators perform mathematical operations on numbers. Most of these will feel intuitively familiar, since they correspond to the basic mathematics you've already encountered.

Addition

Addition uses the `+` symbol, which we've already seen:

```
let nr1 = 12;  
let nr2 = 14;  
let result1 = nr1 + nr2;
```

The same operator can also be used to concatenate strings. Note the trailing space after "Hello" to ensure proper spacing in the result:

```
let str1 = "Hello ";  
let str2 = "addition";  
let result2 = str1 + str2;
```

Logging `result1` and `result2` produces:

```
26  
Hello addition
```

Adding numbers and concatenating strings produce fundamentally different kinds of results. Combining two strings simply joins them together.

Practice exercise 2.2

Create a variable for your name, another for your age, and another indicating whether you can code in JavaScript. Log the following sentence to the console, substituting in your variables:

"Hello, my name is Maaike, I am 29 years old and I can code JavaScript: true."

Subtraction

Subtraction uses the `-` symbol and behaves as expected:

```
let nr1 = 20;
let nr2 = 4;
let str1 = "Hi";
let nr3 = 3;
let result1 = nr1 - nr2;
let result2 = str1 - nr3;
console.log(result1, result2);
```

Output:

```
16 NaN
```

The first result is straightforward arithmetic. The second is more interesting: subtracting a number from a non-numeric string produces `NaN` rather than an error — JavaScript simply reports that the operation doesn't make sense numerically.

Multiplication

Multiplication uses the `*` symbol. Unlike addition, multiplying a number with a non-numeric string always produces `NaN`:

```
let nr1 = 15;
let nr2 = 10;
let str1 = "Hi";
let nr3 = 3;
let result1 = nr1 * nr2;
let result2 = str1 * nr3;
console.log(result1, result2);
```

Output:

150 NaN

Division

Division uses the `/` symbol:

```
let nr1 = 30;  
let nr2 = 5;  
let result1 = nr1 / nr2;  
console.log(result1);
```

Output:

6

Exponentiation

Exponentiation raises a base number to a given power — x^y , read as "x to the power of y," meaning x is multiplied by itself y times. JavaScript uses `**` for this operation:

```
let nr1 = 2;  
let nr2 = 3;  
let result1 = nr1 ** nr2;  
console.log(result1);
```

Output:

8

This computes 2 to the power of 3 ($2 \times 2 \times 2$), which equals 8. We won't dive into a full mathematics lesson here, but it's worth noting that roots can also be computed using fractional exponents — the square root of a value, for instance, is equivalent to raising it to the power of 0.5.

Modulus

Modulus often requires a bit of explanation. It computes the remainder left over after dividing one number by another as many whole times as possible — how many times the divisor fits is irrelevant; only the leftover amount matters. The symbol used is `%`. A few examples:


```
let nr1 = 10;
let nr2 = 3;
let result1 = nr1 % nr2;
console.log(`${nr1} % ${nr2} = ${result1}`);

let nr3 = 8;
let nr4 = 2;
let result2 = nr3 % nr4;
console.log(`${nr3} % ${nr4} = ${result2}`);

let nr5 = 15;
let nr6 = 4;
let result3 = nr5 % nr6;
console.log(`${nr5} % ${nr6} = ${result3}`);
```

Output:

```
10 % 3 = 1
8 % 2 = 0
15 % 4 = 3
```

In the first case, 3 fits into 10 exactly three times, leaving a remainder of 1. In the second, 2 fits into 8 exactly four times with nothing remaining, giving 0. In the third, 4 fits into 15 three times, leaving 3.

This kind of calculation happens automatically in your head all the time. If asked to add 125 minutes to the current time, you'd intuitively use integer division to determine how many whole hours fit into 125 minutes, and then use 125 modulo 60 to find the remaining minutes. Starting at 09:59, you'd add 2 hours to reach 11:59, then add the remaining 5 minutes — but since 59 + 5 again exceeds 60, you'd perform another modulus step, adding one more hour and arriving at a final time of 12:04.

Unary operators: increment and decrement

The final two arithmetic operators — increment and decrement — may be unfamiliar if you're new to programming or coming from a language that doesn't use this notation. A term worth knowing here is operand: the value an operator acts on. In the expression `x + y`, both `x` and `y` are operands.

Increment and decrement each require only a single operand, which is why they're called unary operators. The expression `x++` is shorthand for `x = x + 1`. Similarly, `x--` is shorthand for `x = x - 1`:

```
let nr1 = 4;
nr1++;
console.log(nr1);

let nr2 = 4;
nr2--;
console.log(nr2);
```

Output:

```
5
3
```

Prefix and postfix operators

The increment operator can appear after the operand (`x++`), known as the postfix form, or before it (`++x`), known as the prefix form — and these two forms behave differently in subtle but important ways. Don't worry if the following explanation takes a few readings to fully absorb.

With the postfix form, the current value of the variable is used first, and the increment is applied afterward. In the following example, `nr` is incremented after it's logged the first time, so the first log statement still shows the original value:

```
let nr = 2;
console.log(nr++);
console.log(nr);
```

Output:

```
2
3
```

With the prefix form, the increment happens before the value is used — this is often the behavior you actually want:

```
let nr = 2;
console.log(++nr);
```

Output:

```
3
```

If you can work out what the following snippet logs, you've genuinely grasped the distinction:

```
let nr1 = 4;
let nr2 = 5;
let nr3 = 2;
console.log(nr1++ + ++nr2 * nr3++);
```

This logs **16**. Following standard mathematical order of operations, multiplication happens first, using **6** (the prefix-incremented **nr2**, so it's incremented before multiplying) and **2** (the postfix **nr3**, which is only incremented after being used here, so it doesn't affect this calculation). That multiplication yields **12**. Finally, **nr1** — a postfix operator — contributes its pre-increment value of **4** to the addition, giving a final total of **16**.

Combining the operators

Operators can be combined into more complex expressions, following an order governed by operator precedence rather than simple left-to-right evaluation.

Grouping, using parentheses (and), takes the highest precedence — everything inside parentheses is evaluated first. After that, evaluation order follows operator type, from highest to lowest precedence; operators of equal precedence are evaluated left to right.

Name	Symbol	Example
Grouping	(...)	(x + y)
Exponentiation	**	x ** y
Prefix increment and decrement	--, ++	--x, ++y
Multiplication, division, modulus	*, /, %	x * y, x / y, x % y
Addition and subtraction	+, -	x + y, x - y

Practice exercise 2.3

Write code to calculate the hypotenuse of a right-angled triangle using the Pythagorean theorem, given the lengths of the other two sides. The theorem states that $a^2 + b^2 = c^2$.

Use `prompt()` to obtain values for **a** and **b** from the user. Square both values, add them together, then take the square root to find **c**. Log the result to the console.

*The Pythagorean theorem applies only to right-angled triangles. The two sides meeting at the right angle are called the adjacent and opposite sides — **a** and **b** in the formula. The longest side, opposite the right angle, is called the hypotenuse — **c**.*

Assignment operators

We've already encountered one assignment operator: `=`, used when assigning values to variables. Several other assignment operators exist as shorthand for combining an arithmetic operation with assignment. For instance, `x += 5` means `x = x + 5`, and `x **= 3` means `x = x ** 3`.

Let's trace through an example. We begin by declaring `x` with an initial value of 2:

```
let x = 2;  
x += 2;
```

After this, `x` becomes 4, since `x += 2` is equivalent to `x = x + 2`.

Next, we subtract 2:

```
x -= 2;
```

`x` returns to 2 (`x = x - 2`).

Now we multiply by 6:

```
x *= 6;
```

`x` becomes 12 (`x = x * 6`).

Next, we divide by 3:

```
x /= 3;
```

`x` becomes 4.

Then we apply exponentiation:

```
x **= 2;
```

`x` becomes 16, since 4 raised to the power of 2 equals 16 (4×4).

Finally, the modulus assignment operator:

```
x %= 3;
```

`x` becomes 1, since 3 fits into 16 five times, leaving a remainder of 1.

Practice exercise 2.4

Create three variables, `a`, `b`, and `c`. Update them using assignment operators as follows:

- Add `b` to `a`.
- Divide `a` by `c`.

- Replace `c` with the modulus of `c` and `b`.
 - Print all three values to the console.
-

Comparison operators

Comparison operators differ from the operators discussed so far in that their result is always a Boolean — `true` or `false`.

Equal

Several equality operators determine whether two values are equal, in two distinct flavors: equal value only, or equal value and type.

```
let x = 5;  
let y = "5";  
console.log(x == y);
```

The double-equals operator (`==`) checks only the value, ignoring type differences. Since both `x` and `y` hold the value `5`, this logs `true`. This is sometimes called loose equality.

The triple-equals operator (`===`) checks both value and type:

```
console.log(x === y);
```

Since the types differ (number versus string), this logs `false`. This is called strict equality, and it's generally the comparison you should use, since it's the only one that guarantees both variables are genuinely equal in every respect.

Not equal

Not-equal operators work the same way in reverse — they return `true` when values differ and `false` when they match:

```
let x = 5;  
let y = "5";  
console.log(x != y);
```

This logs `false`, since `x != y` performs a loose comparison: the values match (ignoring type), so they're considered equal, making "not equal" `false`.

```
console.log(x !== y);
```

This strict comparison logs `true`, since the differing types mean `x` and `y` are not strictly equal.

Greater than and smaller than

The greater-than operator (`>`) returns `true` when the left-hand value exceeds the right-hand value. The greater-than-or-equal operator (`>=`) does the same but also returns `true` when the two values are equal.

```
let x = 5;
let y = 6;

console.log(y > x); // true, since y exceeds x
console.log(x > y); // false, since x does not exceed y
console.log(y > y); // false, since y does not exceed itself
console.log(y >= y); // true, since y equals itself
```

The smaller-than (`<`) and smaller-than-or-equal (`<=`) operators behave analogously:

```
console.log(y < x); // false, since y is not smaller than x
console.log(x < y); // true, since x is smaller than y
console.log(y < y); // false, since y is not smaller than itself
console.log(y <= y); // true, since y equals itself
```

Logical operators

Logical operators allow you to check multiple conditions together, or to negate a condition entirely. JavaScript provides `and`, `or`, and `not`.

And

To check whether two conditions are both true — for instance, whether `x` is less than `y` and `y` is less than `z` — use the `&&` operator. The combined expression is `true` only if both sides are individually `true`:

```
let x = 1;
let y = 2;
let z = 3;

console.log(x < y && y < z); // true: both conditions hold
console.log(x > y && y < z); // false: the first condition fails
```

Or

To check whether at least one of two conditions is true, use `||`. The combined expression evaluates to `true` if either side (or both) is `true`:

```
console.log(x > y || y < z); // true: the second condition holds
```

```
console.log(x > y || y > z); // false: neither condition holds
```

Not

To negate a Boolean value — flipping it to its opposite — use the exclamation mark (!):

```
let x = false;  
console.log(!x); // true
```

You can also negate the result of an expression, provided you group it with parentheses so it's fully evaluated before negation is applied:

```
let x = 1;  
let y = 2;  
console.log(!(x < y));
```

Since `x` is less than `y`, the inner expression evaluates to `true`, but the leading `!` flips this to `false` in the final output.

You said: Summary In this chapter, we dealt with the first two programming building blocks: variables and operators.

Summary In this chapter, we dealt with the first two programming building blocks: variables and operators. Variables are special fields that have a name and contain values. We declare a variable by using one of the special variable-defining words: `let`, `var`, or `const`. Variables enable us to make our scripts dynamic, store values, access them later, and change them later. We discussed some primitive data types, including strings, numbers, Booleans, and Symbols, as well as more abstract types such as `undefined` and `null`. You learned how to determine the type of a variable using the `typeof` word. And you saw how you can convert the data type by using the built-in JavaScript methods `Number()`, `String()`, and `Boolean()`. JavaScript Essentials [46] Then we moved on and discussed operators. Operators enable us to work with our variables. They can be used to perform calculations, compare variables, and more. The operators we discussed included arithmetic operators, assignment operators, comparison operators, and logical operators. After this chapter, you are ready to deal with more complex data types, such as arrays and objects. We'll cover these in the next chapter.

Summary

In this chapter, we worked through the first two essential programming building blocks: variables and operators. Variables are named fields that hold values, declared using one of three keywords — `let`, `var`, or `const`. They give our scripts dynamic behavior, letting us store, retrieve, and later modify values as the program runs.

We covered several primitive data types, including strings, numbers, Booleans, and Symbols, along with the more abstract `undefined` and `null` types. You learned how to

determine a variable's type using the `typeof` operator, and how to explicitly convert between types using the built-in `Number()`, `String()`, and `Boolean()` functions.

From there, we turned to operators, which allow us to work with our variables — performing calculations, making comparisons, and more. We covered arithmetic operators, assignment operators, comparison operators, and logical operators.

With this chapter behind you, you're ready to move on to more complex data types — arrays and objects — which we'll explore in the next chapter.